Handout #15
Thursday, 8 October 2026



Autumn Quarter 2026

Problem Set 3

Computer Files Provided for this Problem Set

You can find the following MATLAB files on the AA 279B Canvas website in the "Problem Sets and Solutions" folder:

`AA279lambert_vallado_u.m`	Vallado's Lambert solver using Universal Variables, slightly modified
`AA279jacobianshoot.m`	Sample file for computing Jacobian for shooting method

3.1) (10 points) Given the beginning (1) and ending (2) positions of an orbital transfer:

$$\underline{r_1} = \begin{bmatrix} 0 \\ 0 \\ +3 \end{bmatrix} [DU] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} 0 \\ -4 \\ 0 \end{bmatrix} [DU]$$

and assuming that this is a two body problem, what is the eccentricity of the minimum energy ellipse that contains these two points? Use canonical units, where the gravitational parameter turns out to be $\mu = 1$ $[DU^3/TU^2]$.

3.2) (25 points total) Here you'll use a shooting method to adapt a Lambert solver to handle perturbations such as a third body, oblateness (J_2), air drag, etc. For an orbital transfer near the Earth (μ_{Earth} = 398,600.4418 [km^3/sec^2]), you are given

$$\Delta t = 75\ [min] \quad \text{and} \quad \underline{r_1} = \begin{bmatrix} -15{,}000 \\ 0 \\ 0 \end{bmatrix} [km] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} -2{,}000 \\ -11{,}000 \\ 0 \end{bmatrix} [km]$$

a) (3 points) What is $\underline{v_1}$ according to the Vallado Lambert solver? Assume short-way solutions with no extra revolutions.

b) (3 points) Use MATLAB's **ode113** propagator with **options = odeset('RelTol', 1e-6, 'AbsTol', 1e-9)** to simulate a 75 [min] trajectory using $\underline{r_1}$ and $\underline{v_1}$ as initial conditions. This will require writing a simple state derivative function with 1/r^2 gravity only. Plot the trajectory in 3D to verify it really does connect $\underline{r_1}$ to $\underline{r_2}$ (you don't have to hand in this plot). Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector. Did the Lambert solver do a good job?

c) (3 points) Update your state derivative function to include an additional perturbing oblate gravity field with oblateness coefficient J_{2earth} = 1.0826267x10^{-3} and mean equatorial radius R_{earth} = 6,378.137 [km]. The following lines of MATLAB code compute that perturbation:

```
% Acceleration due to oblateness
normr = norm(r);
rhat = r/normr;
khat = [0 0 1]';
z = r(3);
accJ2 = - (mu*J2*requatorial*requatorial/2)* ...
       ((6*z/(normr^5))*khat ...
      + ((3/(normr^4)) - (15*z*z/(normr^6)))*rhat);

% NOTE: (...) continues statement to next line
```

Rerun the 75 [min] simulation with oblateness included. Use the $\underline{v_1}$ found in part (a) in your initial conditions. Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector.

d) (10 points) Assume that you want all the error components of the "miss distance" to be less than 10 [cm] (admittedly, this is probably overkill). Use a shooting method to perform one update step based on the error found in part (c). What is the updated $\underline{v_1}$? HINTS: You can use the provided **`AA279jacobianshoot.m`** function (be sure to read the comments) or one of your own to evaluate the Jacobian using finite differences. A small velocity increment of 0.01 [km/sec] or 0.001 [km/sec] is a reasonable place to start in numerically evaluating the Jacobian.

e) (3 points) Run the 75 [min] simulation with the updated value of $\underline{v_1}$. What is the new position error vector at the end?

f) (3 points) How many iterations are required to obtain a position error with all components less than 10 [cm]? What is the final value of $\underline{v_1}$ you obtained with the shooting method?

3.3) (65 points total) In this problem you'll analyze ΔV requirements for a system that fires a launch vehicle from a fixed launcher to intercept or rendezvous with a satellite already in orbit, mirroring what we did in class and the analysis in Section 5.7 of Bate, Mueller, and White. Along the way, you'll use some of the computer code you've used for previous space mechanics classes. (Specifically, you'll need to use Greenwich Sidereal Time to properly orient the Earth to find launcher position and velocity, and you'll need to be able to solve Kepler's equation to find a satellite's position and velocity at any time given its orbital elements.)

In November 2013, Planet Labs (based in San Francisco, CA) successfully orbited two cubesats (Dove 3 and Dove 4) from a Ukrainian Dnepr rocket. Pretend that in early 2014 one of Planet Labs' rivals, Skybox Imaging (then based in Mountain View, CA, later acquired by Google, renamed Terra Bella, and even later acquired by Planet!) decided it would like to either destroy or retrieve Planet Labs' Dove 3 satellite for competitive analysis. Imagine you were one of Skybox's interns in early 2014 and that you were assigned the task of determining ΔV tradeoffs for a launch system that could intercept or rendezvous with Dove 3.

Earth Data

Assume the Earth is a sphere (i.e. no oblateness), both when modeling its gravity field and when translating latitude/longitude/elevation to ECEF coordinates

μ_{Earth} = 398600.4418 [km^3/sec^2] gravitational parameter
r_{Earth} = 6378.137 [km] radius
ω_{Earth} = 0.0000729211585530 [rad/sec] rotation rate in inertial space

Greenwich sidereal time θ_g at the beginning of 2014 (0000h 1 January 2014 UTC) was 6.7045617 [sidereal hours].

Launcher Data

The launcher is located at Skybox's headquarters in Mountain View, CA:

longitude = -122.081944 [deg] (Western hemisphere)
latitude = +37.389444 [deg] (Northern hemisphere)
elevation above Earth sphere = 11 [m]

Satellite Data

The following is a set of orbital elements for Dove 3 in early 2014:

Year of epoch = 2014
t_0 = 21.68857885 [days] Time of epoch as day of year plus fraction (t_0 is in late-January)
i = 97.7791 [deg]
Ω = 95.3093 [deg]
e = 0.0155616
ω = 357.8679 [deg]
M_0 = 2.1812 [deg]
n = 14.56715041 [rev/day]

NOTE: t_0 is expressed in this format as "day of year plus fraction." For example, noon on January 1st would be expressed as 1.5 [days], not 0.5 [days]. Keep this in mind when writing your code…

Simulation Period

Your simulation should start at 0100h on 23 January 2014 UTC. Search a design space of two variables. The first variable is the delay between the simulation start time and the launch, i.e. "launch time." The second variable is the "time of flight" (TOF) of the launch vehicle. The first variable should range from 0 [hr] to 6 [hr], and the second should range from 0 [hr] to 2 [hr].

Additional Items

- Use short way solutions to the Lambert Problem and no multiple revolutions.
- Assume the rocket is launched directly off the surface of the earth with no gravity drag or atmospheric drag. A ΔV is applied at the launcher, and the launch vehicle flies ballistically from there.
- As discussed in lecture, before each Lambert evaluation, check that the requested transfer time doesn't force a parabolic or hyperbolic path. If it does, don't solve the Lambert Problem; just flag that point as being rejected.
- Use Vallado's algorithm discussed in lecture to flag any Lambert solutions that result in the launch vehicle's hitting the Earth before arriving at the target.
- To reduce debugging time, you may want to run on a very sparse grid at first and then move to denser and denser grids.
- If you are having trouble getting parts (a) and (b) working, we can provide some AA 279A notes that may help.

a) (3 points) Write code that computes position and inertial velocity of the launcher at a given time in ECI coordinates.

NOTE: This part requires you to find a rotation matrix for converting between Earth Centered Earth Fixed (ECEF) and Earth Centered Inertial (ECI) coordinates, which will in turn require computing the Greenwich sidereal time θ_g at any given time. In AA 279A or its equivalent, you may have done this using website lookups and/or detailed computations involving Modified Julian Date, DUT1, and UT1. For this problem set, we're asking you to do something much simpler. To find the θ_g at a given time, use

$$\theta_g(t) = \theta_g(t_{ref}) + \omega_{Earth}\,(t - t_{ref})$$

where

$\theta_g(t)$ is in [rad] .

$\theta_g(t_{ref})$ is the reference value we gave you for 0000h 1 January 2014 and needs to be converted from [sidereal hours] to [rad] using 24 [sidereal hours] = 2π [rad] . Don't forget that "sidereal hours" are simply a unit of angle measurement.

ω_{Earth} is in [rad/sec] , and

(t - t_{ref}) is the number of seconds between 0000h 1 January 2014 and the launch time t .

b) (6 points) Write code that computes position and inertial velocity of a satellite at a given time in ECI coordinates given its orbital elements.

c) (20 points) For one-burn intercepts, create a total ΔV plot (MATLAB's **`contour`** or **`contourf`** functions may help) with launch time on the horizontal axis, and time of flight (TOF) on the vertical axis. Use a grid spacing of 2 [min] or tighter. As you're determining ΔV for each combination, you can flag parabolic/hyberbolic and Earth-intersecting cases with unique/non-physical values in the matrix where you store your ΔV data. Briefly note any interesting features of the intercept ΔV plot.

d) (20 points) Create a total ΔV plot for two-burn rendezvous. Note that this data can be generated and stored inside the same loop that generates the intercept data. Briefly note any interesting features of the rendezvous ΔV plot.

e) (4 points) Within the plotted data, what launch time/TOF combination for intercept minimizes total ΔV? What is this minimum ΔV? Is it possible for this minimum value to be less than a typical circular velocity for LEO?

f) (4 points) Within the plotted data, what launch time/TOF combination for rendezvous minimizes total ΔV? What is this minimum ΔV?

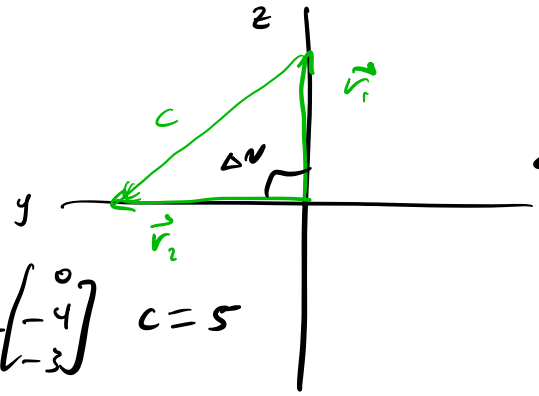
g) (4 points) Assume now that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the earliest possible intercept time (i.e. time after start of simulation)? When does the launch occur for this case?

h) (4 points) Assume again that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the shortest possible time of flight? When does the launch occur for this case?

3.1) (10 points) Given the beginning (1) and ending (2) positions of an orbital transfer:

$$\underline{r_1} = \begin{bmatrix} 0 \\ 0 \\ +3 \end{bmatrix} [DU] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} 0 \\ -4 \\ 0 \end{bmatrix} [DU]$$

and assuming that this is a two body problem, what is the eccentricity of the minimum energy ellipse that contains these two points? Use canonical units, where the gravitational parameter turns out to be $\mu = 1$ $[DU^3/TU^2]$.



$\Delta\nu = 90°$

$\vec{r}_1 = \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$ $r_1 = 3$

$\vec{r}_2 = \begin{bmatrix} 0 \\ -4 \\ 0 \end{bmatrix}$ $r_2 = 4$

$\vec{c} = \begin{bmatrix} 0 \\ -4 \\ -3 \end{bmatrix}$ $c = 5$

$p_m = \frac{r_1 r_2}{c}(1 - \cos\Delta\nu) = \frac{3 \cdot 4}{5} = \frac{12}{5}$ $\quad s = \frac{r_1 + r_2 + c}{2} = \frac{3+4+5}{2} = \frac{12}{2} = 6$

$e_m = \sqrt{1 - \frac{2p_m}{s}} = \sqrt{1 - \frac{2\left(\frac{12}{5}\right)^2}{6}} = \sqrt{1 - \frac{4}{5}} = \sqrt{\frac{1}{5}}$ $\quad e_m \approx 0.447$

3.2) (25 points total) Here you'll use a shooting method to adapt a Lambert solver to handle perturbations such as a third body, oblateness (J_2), air drag, etc. For an orbital transfer near the Earth (μ_{Earth} = 398,600.4418 [km^3/sec^2]), you are given

$$\Delta t = 75\ [min] \quad \text{and} \quad \underline{r_1} = \begin{bmatrix} -15{,}000 \\ 0 \\ 0 \end{bmatrix} [km] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} -2{,}000 \\ -11{,}000 \\ 0 \end{bmatrix} [km]$$

a) (3 points) What is $\underline{v_1}$ according to the Vallado Lambert solver? Assume short-way solutions with no extra revolutions.

b) (3 points) Use MATLAB's **ode113** propagator with **options = odeset('RelTol', 1e-6, 'AbsTol', 1e-9)** to simulate a 75 [min] trajectory using $\underline{r_1}$ and $\underline{v_1}$ as initial conditions. This will require writing a simple state derivative function with 1/r^2 gravity only. Plot the trajectory in 3D to verify it really does connect $\underline{r_1}$ to $\underline{r_2}$ (you don't have to hand in this plot). Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector. Did the Lambert solver do a good job?

c) (3 points) Update your state derivative function to include an additional perturbing oblate gravity field with oblateness coefficient J_{2earth} = 1.0826267x10^{-3} and mean equatorial radius R_{earth} = 6,378.137 [km]. The following lines of MATLAB code compute that perturbation:

```
% Acceleration due to oblateness
normr = norm(r);
rhat = r/normr;
khat = [0 0 1]';
z = r(3);
accJ2 = - (mu*J2*requatorial*requatorial/2)* ...
        ((6*z/(normr^5))*khat ...
      + ((3/(normr^4)) - (15*z*z/(normr^6)))*rhat);

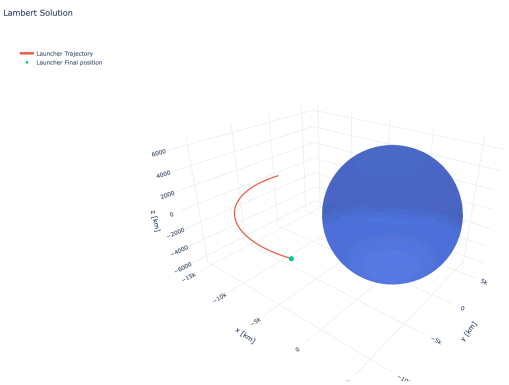
% NOTE: (...) continues statement to next line
```

Rerun the 75 [min] simulation with oblateness included. Use the $\underline{v_1}$ found in part (a) in your initial conditions. Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector.

a)

```
Vallado v1: 4.116082219551546 km/s
```

b)




```
Final r value [km]:
        [ -1999.9895746  -10999.98043675      0.        ]
Goal r value [km]:
        [ -2000 -11000      0]
Err [km]:
        [-0.0104254  -0.01956325  0.        ] (0.0222 km)
Within 22.168m, so it did a good job
```

c)

```
Final r value [km]:
        [ -1994.11058014 -10995.388839        0.        ]
Goal r value [km]:
        [ -2000 -11000      0]
Err [km]:
        [-5.88941986 -4.611161    0.        ] (7.4798 km)
Not particularly well compared to Lambert
```

d) (10 points) Assume that you want all the error components of the "miss distance" to be less than 10 [cm] (admittedly, this is probably overkill). Use a shooting method to perform one update step based on the error found in part (c). What is the updated $\underline{v_1}$? HINTS: You can use the provided **AA279jacobianshoot.m** function (be sure to read the comments) or one of your own to evaluate the Jacobian using finite differences. A small velocity increment of 0.01 [km/sec] or 0.001 [km/sec] is a reasonable place to start in numerically evaluating the Jacobian.

e) (3 points) Run the 75 [min] simulation with the updated value of $\underline{v_1}$. What is the new position error vector at the end?

f) (3 points) How many iterations are required to obtain a position error with all components less than 10 [cm]? What is the final value of $\underline{v_1}$ you obtained with the shooting method?

d)

```
Old v1 value [km/s]:
        [-0.65513692 -4.06361027  0.        ] (4.116 km/s)
Final v1 value [km/s]:
        [-6.55880979e-01 -4.06399725e+00  5.80741636e-31] (4.117 km/s)

Err [m/s]:
        [ 7.44054505e-01  3.86975119e-01 -5.80741636e-28] (0.8387 m/s)
Very small difference between original and 1 iteration
```

```
Converged in 3 iterations with:
        dr = 8.98310e-11 km
```

```
One-step v1 value [km/s]:
        [-6.55880979e-01 -4.06399725e+00  5.80741636e-31] (4.11658274 km/s)
Final Updated v1 value [km/s]:
        [-6.55880936e-01 -4.06399751e+00  5.40032976e-34] (4.11658300 km/s)

Difference [km/s]:
        [-4.34170663e-08  2.62738334e-07  5.80201603e-31], (2.663e-07 km/s)
```

3.3) (65 points total) In this problem you'll analyze ΔV requirements for a system that fires a launch vehicle from a fixed launcher to intercept or rendezvous with a satellite already in orbit, mirroring what we did in class and the analysis in Section 5.7 of Bate, Mueller, and White. Along the way, you'll use some of the computer code you've used for previous space mechanics classes. (Specifically, you'll need to use Greenwich Sidereal Time to properly orient the Earth to find launcher position and velocity, and you'll need to be able to solve Kepler's equation to find a satellite's position and velocity at any time given its orbital elements.)

In November 2013, Planet Labs (based in San Francisco, CA) successfully orbited two cubesats (Dove 3 and Dove 4) from a Ukrainian Dnepr rocket. Pretend that in early 2014 one of Planet Labs' rivals, Skybox Imaging (then based in Mountain View, CA, later acquired by Google, renamed Terra Bella, and even later acquired by Planet!) decided it would like to either destroy or retrieve Planet Labs' Dove 3 satellite for competitive analysis. Imagine you were one of Skybox's interns in early 2014 and that you were assigned the task of determining ΔV tradeoffs

Earth Data
Assume the Earth is a sphere (i.e. no oblateness), both when modeling its gravity field and when translating latitude/longitude/elevation to ECEF coordinates

μ_{Earth} = 398600.4418 [km^3/sec^2] gravitational parameter
r_{Earth} = 6378.137 [km] radius
ω_{Earth} = 0.000072921158553O [rad/sec] rotation rate in inertial space

Greenwich sidereal time θ_0 at the beginning of 2014 (0000h 1 January 2014 UTC) was 6.7045617 [sidereal hours].

Launcher Data
The launcher is located at Skybox's headquarters in Mountain View, CA:

longitude = -122.081944 [deg] (Western hemisphere)
latitude = +37.389444 [deg] (Northern hemisphere)
elevation above Earth sphere = 11 [m]

Satellite Data
The following is a set of orbital elements for Dove 3 in early 2014:

Year of epoch = 2014
t_0 = 21.68857885 [days] Time of epoch as day of year plus fraction (t_0 is in late-January)
i = 97.7791 [deg]
Ω = 95.3093 [deg]
e = 0.0155616
ω = 357.8679 [deg]
M_0 = 2.1812 [deg]
n = 14.56715041 [rev/day]

NOTE: t_0 is expressed in this format as "day of year plus fraction." For example, noon on January 1st would be expressed as 1.5 [days], not 0.5 [days]. Keep this in mind when writing your code…

Simulation Period
Your simulation should start at 0100h on 23 January 2014 UTC. Search a design space of two variables. The first variable is the delay between the simulation start time and the launch, i.e. "launch time." The second variable is the "time of flight" (TOF) of the launch vehicle. The first variable should range from 0 [hr] to 6 [hr], and the second should range from 0 [hr] to 2 [hr].

Additional Items

- Use short way solutions to the Lambert Problem and no multiple revolutions.
- Assume the rocket is launched directly off the surface of the earth with no gravity drag or atmospheric drag. A ΔV is applied at the launcher, and the launch vehicle flies ballistically from there.
- As discussed in lecture, before each Lambert evaluation, check that the requested transfer time doesn't force a parabolic or hyperbolic path. If it does, don't solve the Lambert Problem; just flag that point as being rejected.
- Use Vallado's algorithm discussed in lecture to flag any Lambert solutions that result in the launch vehicle's hitting the Earth before arriving at the target.
- To reduce debugging time, you may want to run on a very sparse grid at first and then move to denser and denser grids.
- If you are having trouble getting parts (a) and (b) working, we can provide some AA 279A notes that may help.

a) (3 points) Write code that computes position and inertial velocity of the launcher at a given time in ECI coordinates.

NOTE: This part requires you to find a rotation matrix for converting between Earth Centered Earth Fixed (ECEF) and Earth Centered Inertial (ECI) coordinates, which will in turn require computing the Greenwich sidereal time θ_E at any given time. In AA 279A or its equivalent, you may have done this using website lookups and/or detailed computations involving Modified Julian Date, DUT1, and UT1. For this problem set, we're asking you to do something much simpler. To find the θ_E at a given time, use

$\theta_E(t) = \theta_E(t_{ref}) + \omega_{Earth}\,(t - t_{ref})$

where

$\theta_E(t)$ is in [rad] .

$\theta_E(t_{ref})$ is the reference value we gave you for 0000h 1 January 2014 and needs to be converted from [sidereal hours] to [rad] using 24 [sidereal hours] = 2π [rad] . Don't forget that "sidereal hours" are simply a unit of angle measurement.

ω_{Earth} is in [rad/sec] , and

$(t - t_{ref})$ is the number of seconds between 0000h 1 January 2014 and the launch time t .

b) (6 points) Write code that computes position and inertial velocity of a satellite at a given time in ECI coordinates given its orbital elements.

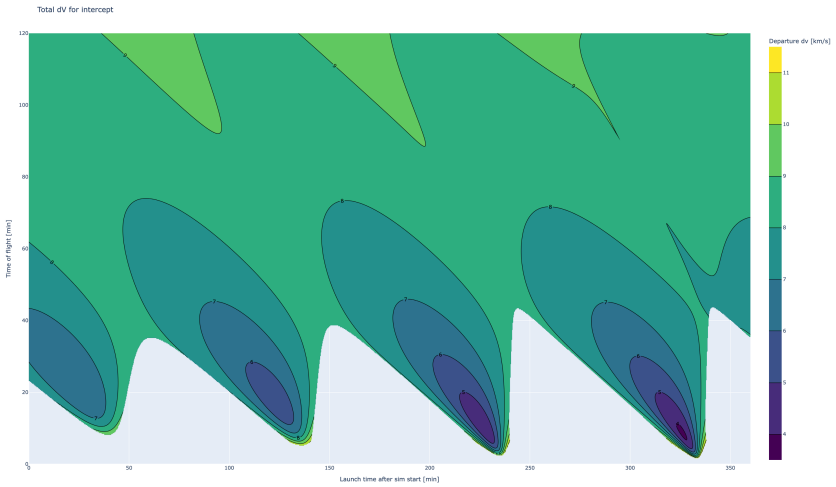
c) (20 points) For one-burn intercepts, create a total ΔV plot (MATLAB's **`contour`** or **`contourf`** functions may help) with launch time on the horizontal axis, and time of flight (TOF) on the vertical axis. Use a grid spacing of 2 [min] or tighter. As you're determining ΔV for each combination, you can flag parabolic/hyberbolic and Earth-intersecting cases with unique/non-physical values in the matrix where you store your ΔV data. Briefly note any interesting features of the intercept ΔV plot.

d) (20 points) Create a total ΔV plot for two-burn rendezvous. Note that this data can be generated and stored inside the same loop that generates the intercept data. Briefly note any interesting features of the rendezvous ΔV plot.

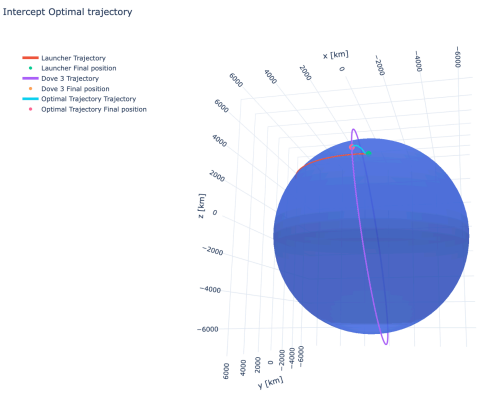
a) see MATLAB

b) see MATLAB

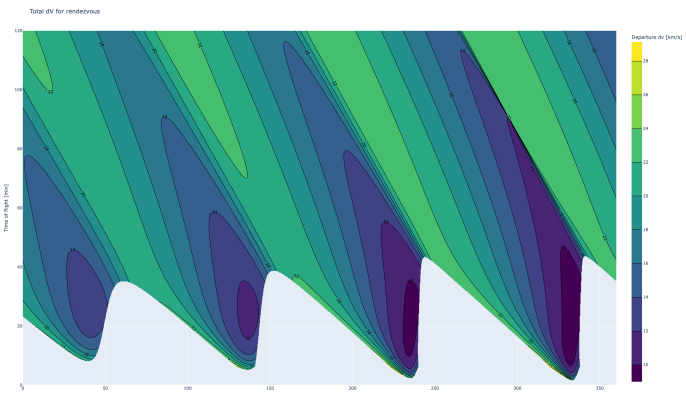
c)

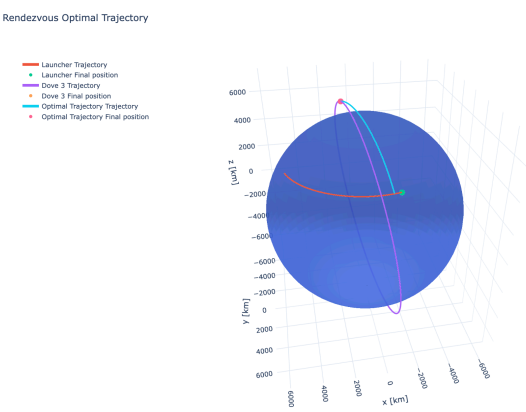


The dv plot seems to have a period of about 90 minutes (typical of a LEO satellite).
The dv in the center of its bulbs also decreases as time goes on.



d)





Looks very similar to the intercept plot, but there are clear diagonal lines that have a lot of dv.
The total dv required in the center of the bulbs also decreases each period.
The lines where the solution becomes invalid (intercepts Earch or parabolic/hyperbolic) are almost vertical close to the minimum dv.
This is likely because the optimal trajectory goes North slowly near the satellite's path, so delaying even a bit more will cause it to miss entirely.

e) (4 points) Within the plotted data, what launch time/TOF combination for intercept minimizes total ΔV? What is this minimum ΔV? Is it possible for this minimum value to be less than a typical circular velocity for LEO?

f) (4 points) Within the plotted data, what launch time/TOF combination for rendezvous minimizes total ΔV? What is this minimum ΔV?

g) (4 points) Assume now that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the earliest possible intercept time (i.e. time after start of simulation)? When does the launch occur for this case?

h) (4 points) Assume again that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the shortest possible time of flight? When does the launch occur for this case?

e/f)

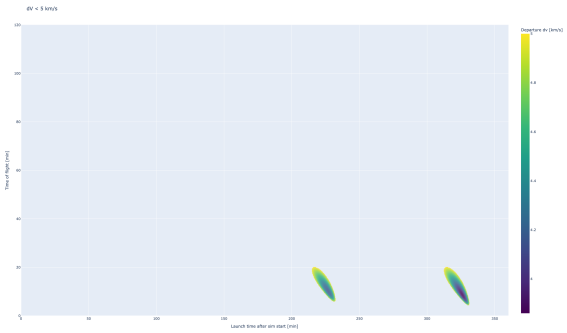
```
Min dv for intercept:
        3.860 km/s at delay of 326.0 min and tof of 8.75 min
Min dv for rendezvous:
        8.984 km/s at delay of 333.75 min and tof of 19.75 min
```

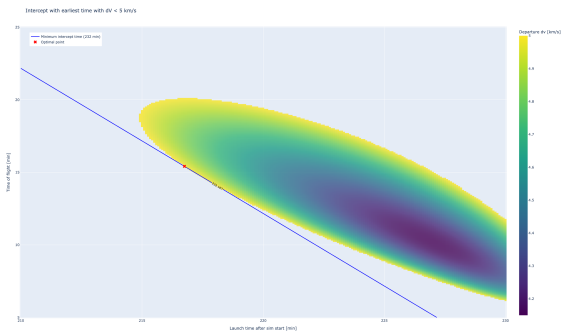
```
It could be less than the circular velocity because the launcher has an initial velocity due to the rotation of the Earth
```

g)

```
Quickest intercept at:
        232.16666666666666 min with delay of 216.75, tof of 15.416666666666666 min, and dv of 4.998 km/s
```

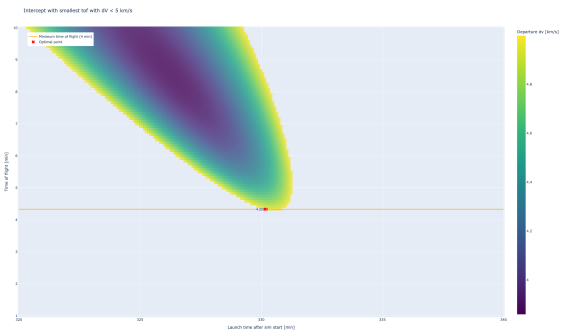
```
Launch time for quickest intercept:
        2014-01-23 04:36:45+00:00
```





h)

```
Smallest tof at:
        334.5 min with delay of 330.1666666666667, tof of 4.333333333333333 min, and dv of 4.992 km/s
```




```
Launch time for smallest time of flight:
        2014-01-23 06:30:10+00:00
```

AA 279B - Homework 3

Imports

In [1]:
```
%load_ext autoreload
%autoreload 2
```

In [2]:
```
import numpy as np
from numpy.linalg import norm
from scipy.integrate import solve_ivp
from scipy.optimize import root_scalar, fsolve
import matplotlib.pyplot as plt
from lamberthub import vallado2013
import plotly.graph_objects as go
from scipy.interpolate import CubicSpline

import astropy.units as u
from astropy.time import Time
from astropy.coordinates import GCRS, ITRS, CartesianRepresentation
```

Problem 3.1

In [3]:
```
# ICs
r1 = np.array([0, 0, 3])
r2 = np.array([0, -4, 0])

# 3-4-5 triangle
r1 = 3
r2 = 4
c = 5
s = (r1 + r2 + c) / 2
p_m = 2/c * (s - r1) * (s - r2)

e_m = np.sqrt(1 - 2*p_m/s)

print(f"ecc: {e_m}")
```

```
ecc: 0.44721359549995776
```

Problem 3.2

In [4]:
```
# km3/m2
MU_EARTH = 398600.4418

# km
R_EARTH = 6378.137

# rad/s
```

```
W_EARTH = 0.0000729211585530

J2_EARTH = 1.0826267e-3

MIN_TO_SEC = 60
tof = 75 * MIN_TO_SEC
```

In [5]:
```
# ICs
r1 = np.array([-15000, 0, 0])
r2 = np.array([-2000, -11000, 0])
```

Part a

In [6]:
```
v1,v2 = vallado2013(MU_EARTH, r1, r2, tof)
print(f"Vallado v1: {norm(v1)} km/s")
```

```
Vallado v1: 4.116082219551546 km/s
```

Part b

In [7]:
```
def two_body(t,x,mu=MU_EARTH):
    del t
    r,v = x[0:3], x[3:6]
    a = -mu / norm(r)**3 * r
    return np.concat((v,a))

N = 1000
t_arr = np.linspace(0, tof, N)
sol: np.ndarray = solve_ivp(two_body, (0, tof), np.concatenate((r1, v1)), t_
```

In [8]:
```
print(f"Final r value [km]: \n\t{sol.y[0:3, -1]}")
print(f"Goal r value [km]: \n\t{r2}")
err = r2-sol.y[0:3,-1]
print(f"Err [km]:\n\t{err} ({norm(err):.4f} km)")
print(f"Within {norm(err)*1000:.3f}m, so it did a good job")
```

```
Final r value [km]:
        [ -1999.9895746  -10999.98043675      0.        ]
Goal r value [km]:
        [ -2000 -11000      0]
Err [km]:
        [-0.0104254  -0.01956325  0.        ] (0.0222 km)
Within 22.168m, so it did a good job
```

In [9]:
```
def plot_3d(
    t_arrs,
    traces,
    labels="Orbit",
    title="3D Orbits",
    r_earth=R_EARTH,
    t_max=None
):

    fig = go.Figure()
```

```
    # Earth
    u_ = np.linspace(0, 2*np.pi, 80)
    v_ = np.linspace(0, np.pi, 40)

    x_earth = r_earth * np.outer(np.cos(u_), np.sin(v_))
    y_earth = r_earth * np.outer(np.sin(u_), np.sin(v_))
    z_earth = r_earth * np.outer(np.ones_like(u_), np.cos(v_))

    fig.add_trace(go.Surface(
        x=x_earth,
        y=y_earth,
        z=z_earth,
        name="Earth",
        showscale=False,
        opacity=0.7,
        colorscale=[[0, "royalblue"], [1, "royalblue"]]
    ))

    # Allow a single (N,3) trajectory
    if isinstance(traces, np.ndarray) and traces.ndim == 2:
        traces = [traces]

    # One time array shared by all trajectories
    if isinstance(t_arrs, np.ndarray) and t_arrs.ndim == 1:
        t_arrs = [t_arrs] * len(traces)

    if isinstance(labels, str):
        labels = [labels] * len(traces)

    for t_arr, trace, label in zip(t_arrs, traces, labels):
        t_arr = np.asarray(t_arr)
        trace = np.asarray(trace)

        if trace.ndim != 2 or trace.shape[1] != 3:
            raise ValueError(
                f"{label}: trace must have shape (N, 3), got {trace.shape}"
            )

        if len(t_arr) != len(trace):
            raise ValueError(
                f"{label}: len(t_arr)={len(t_arr)} != len(trace)={len(trace)
            )

        # Clip to requested maximum time
        if t_max is not None:
            mask = t_arr <= t_max
            t_arr = t_arr[mask]
            trace = trace[mask]

        if len(t_arr) < 2:
            continue

        cs = CubicSpline(t_arr, trace, axis=0)

        t_fine = np.linspace(t_arr[0], t_arr[-1], 1000)
```

```
        r_fine = cs(t_fine)

        fig.add_trace(go.Scatter3d(
            x=r_fine[:, 0],
            y=r_fine[:, 1],
            z=r_fine[:, 2],
            mode="lines",
            name=f"{label} Trajectory",
            line=dict(width=5)
        ))

        fig.add_trace(go.Scatter3d(
            x=[r_fine[-1, 0]],
            y=[r_fine[-1, 1]],
            z=[r_fine[-1, 2]],
            mode="markers",
            name=f"{label} Final position",
            marker=dict(size=6)
        ))

    fig.update_layout(
        title=title,
        scene=dict(
            xaxis_title="x [km]",
            yaxis_title="y [km]",
            zaxis_title="z [km]",
            aspectmode="data"
        ),
        template="plotly_white",
        legend=dict(x=0.02, y=0.98),
        height=900
    )

    fig.show()
```

In [10]:
```
plot_3d(sol.t, sol.y[:3,:].T, "Launcher", title="Lambert Solution")
```

In [11]:
```
fig, ax = plt.subplots(figsize=(20, 12))
ax.set_title('Lambert Solution and Propagation')

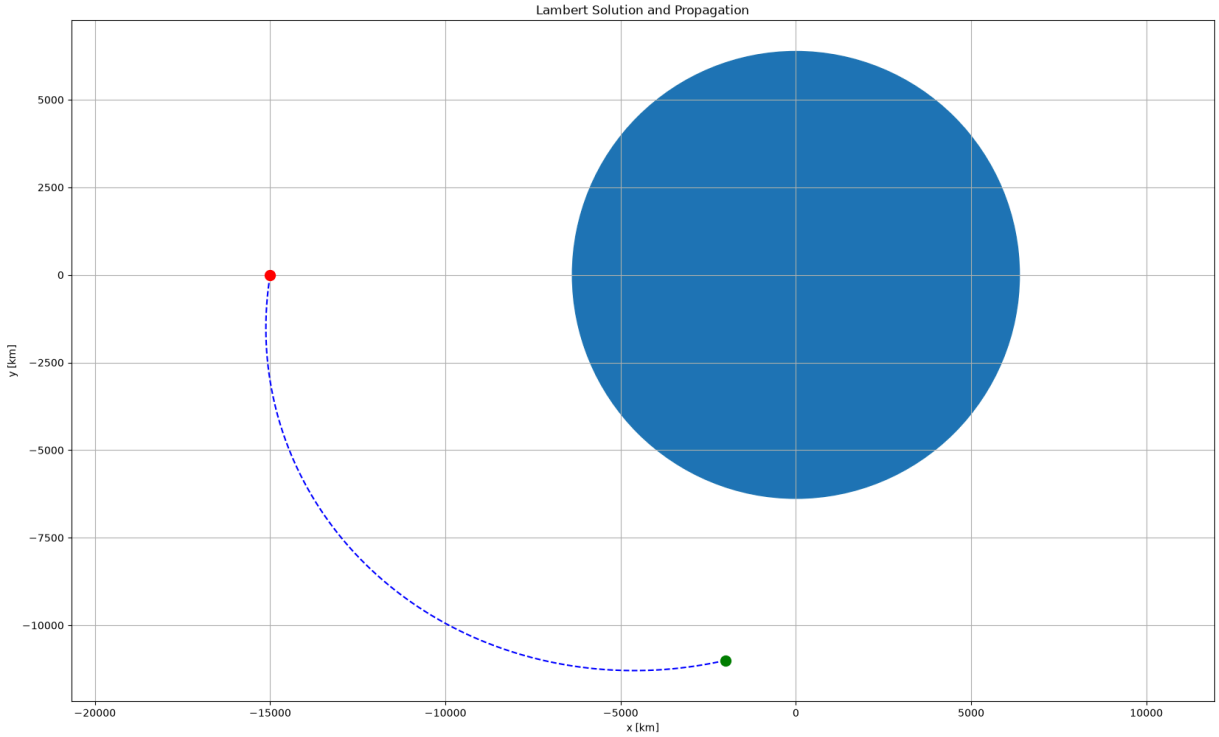
# Trajectory
ax.plot(sol.y[0], sol.y[1], 'b--', label='Trajectory')

# Points
ax.plot(r1[0], r1[1], 'ro', markersize=10, label='r1')
ax.plot(r2[0], r2[1], 'go', markersize=10, label='r2')

# Body
ax.add_patch(plt.Circle((0,0), R_EARTH, color='tab:blue', fill=True))


ax.set_xlabel('x [km]')
ax.set_ylabel('y [km]')
ax.set_aspect('equal', adjustable='datalim')
```

```
ax.grid(True)
plt.show()
```



Part c

In [12]:
```
def two_body_with_J2(t,x,mu=MU_EARTH, j2=J2_EARTH):
    state_dot = two_body(t,x,mu)

    norm_r = norm(x[0:3])
    r_hat = x[0:3] / norm_r
    k_hat = np.array([0,0,1])
    z = x[2]

    a_J2 = -(mu*j2*R_EARTH*R_EARTH/2) * (
        6*z/norm_r**5 * k_hat +
        (3/norm_r**4 - 15*z*z/norm_r**6) * r_hat
    )
    state_dot[3:6] += a_J2
    return state_dot


sol_J2: np.ndarray = solve_ivp(two_body_with_J2, (0, tof), np.concatenate((r
```

In [13]:
```
print(f"Final r value [km]: \n\t{sol_J2.y[0:3, -1]}")
print(f"Goal r value [km]: \n\t{r2}")

err = r2-sol_J2.y[0:3,-1]
print(f"Err [km]:\n\t{err} ({norm(err):.4f} km)")
print("Not particularly well compared to Lambert")
```

```
Final r value [km]:
        [ -1994.11058014 -10995.388839        0.        ]
Goal r value [km]:
        [ -2000 -11000      0]
Err [km]:
        [-5.88941986 -4.611161    0.        ] (7.4798 km)
Not particularly well compared to Lambert
```

In [14]:
```
fig, ax = plt.subplots(figsize=(20, 12))
ax.set_title('Lambert Solution and Propagation with J2')

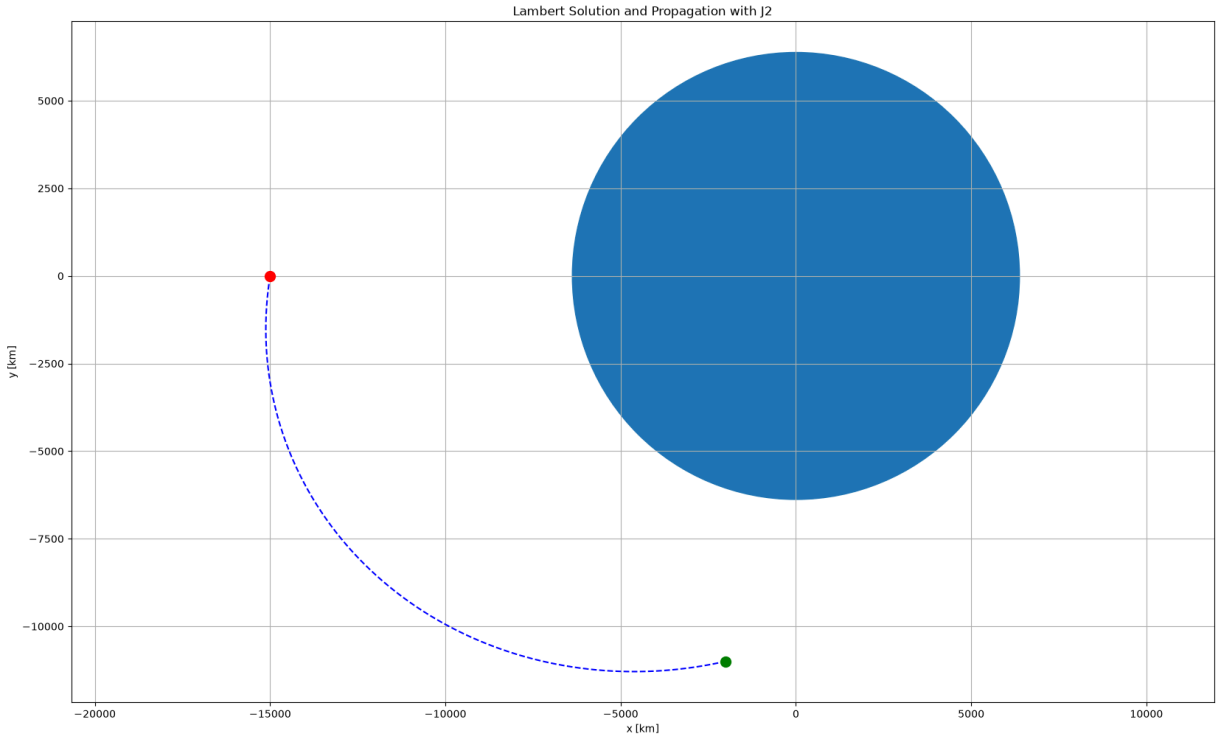
# Trajectory
ax.plot(sol_J2.y[0], sol_J2.y[1], 'b--', label='Trajectory')

# Points
ax.plot(r1[0], r1[1], 'ro', markersize=10, label='r1')
ax.plot(r2[0], r2[1], 'go', markersize=10, label='r2')

# Body
ax.add_patch(plt.Circle((0,0), R_EARTH, color='tab:blue', fill=True))


ax.set_xlabel('x [km]')
ax.set_ylabel('y [km]')
ax.set_aspect('equal', adjustable='datalim')

ax.grid(True)
plt.show()
```



Part d

In [15]:
```
def AA279jacobianshoot(odefun, tspan, y0in, dv, t_arr):
    """
    Returns 3x3 Jacobian for shooting method

    AA279 Function Library
    Last modified: 7 October 2026 by Andrew K. Barrows

    Jacobian describes sensitivity of final position to initial velocity

    State must be of the form [rx ry rz vx vy vz]'

    sample function call:
    J = jacobianshoot(@earthorbitbody, [0; 75*60], [r1; v1], 0.001)

    odefun function handle for state derivative function
    tspan trajectory start and end times [sec]
    y0in state at which J should be evaluated
    dv small velocity increment used for finite differences
    t_arr array to be evaluated at

    """

    J = np.zeros((3,3))
    y0in = np.asarray(y0in)

    # vx
    y0 = y0in + [0,0,0,dv,0,0]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rhigh = sol.y[0:3, -1]
    y0 = y0in + [0,0,0,-dv,0,0]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rlow = sol.y[0:3, -1]
    J[:, 0] = (rhigh-rlow)/(2*dv)

    # vx
    y0 = y0in + [0,0,0,0,dv,0]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rhigh = sol.y[0:3, -1]
    y0 = y0in + [0,0,0,0,-dv,0]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rlow = sol.y[0:3, -1]
    J[:, 1] = (rhigh-rlow)/(2*dv)

    # vx
    y0 = y0in + [0,0,0,0,0,dv]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rhigh = sol.y[0:3, -1]
    y0 = y0in + [0,0,0,0,0,-dv]
    sol = solve_ivp(odefun, (t_arr[0], t_arr[-1]), y0, method='LSODA', t_eva
    rlow = sol.y[0:3, -1]
    J[:, 2] = (rhigh-rlow)/(2*dv)

    return J
```

In [16]:
```
def find_initial_conditions(t_arr, ode, state0, r_goal, iter = 10, converger
    """10 cm bound"""


    tof = t_arr[-1]

    state = state0

    for i in range(iter):
        sim = solve_ivp(ode, (0, tof), state, method='LSODA', t_eval=t_arr,

        dr = sim[0:3,-1] - r_goal
        dr_norm = norm(dr)

        if verbose:
            print(f"dr: {dr_norm:.5f}km")

        if norm(dr) < convergence_bound:
            print(f"\nConverged in {i} iterations with:\n\tdr = {dr_norm:.5e
            break

        J = AA279jacobianshoot(ode, (0, tof), state, dv=0.001, t_arr=t_arr)

        # Only adjusting velocity
        state[3:6] -= np.linalg.pinv(J) @ dr

    return state

state0_one_update = find_initial_conditions(t_arr, two_body_with_J2, np.conc
v1_one_update = state0_one_update[3:6]
```

In [17]:
```
print(f"Old v1 value [km/s]: \n\t{v1} ({norm(v1):.3f} km/s)")
print(f"Final v1 value [km/s]: \n\t{v1_one_update} ({norm(v1_one_update):.3f
# print(f"Difference [km/s]:\n\t{v1-v1_one_update}")
print()
diff_m = (v1-v1_one_update)*1000
print(f"Err [m/s]:\n\t{diff_m} ({norm(diff_m):.4f} m/s)")
print("Very small difference between original and 1 iteration")
```

```
Old v1 value [km/s]:
        [-0.65513692 -4.06361027  0.        ] (4.116 km/s)
Final v1 value [km/s]:
        [-6.55880979e-01 -4.06399725e+00  5.80741636e-31] (4.117 km/s)

Err [m/s]:
        [ 7.44054505e-01  3.86975119e-01 -5.80741636e-28] (0.8387 m/s)
Very small difference between original and 1 iteration
```

Part e

In [18]:
```
sol_one_update: np.ndarray = solve_ivp(two_body_with_J2, (0, tof), state0_or
```

In [19]:
```
fig, ax = plt.subplots(figsize=(20, 12))
ax.set_title('Lambert Solution and Propagation with J2')
```

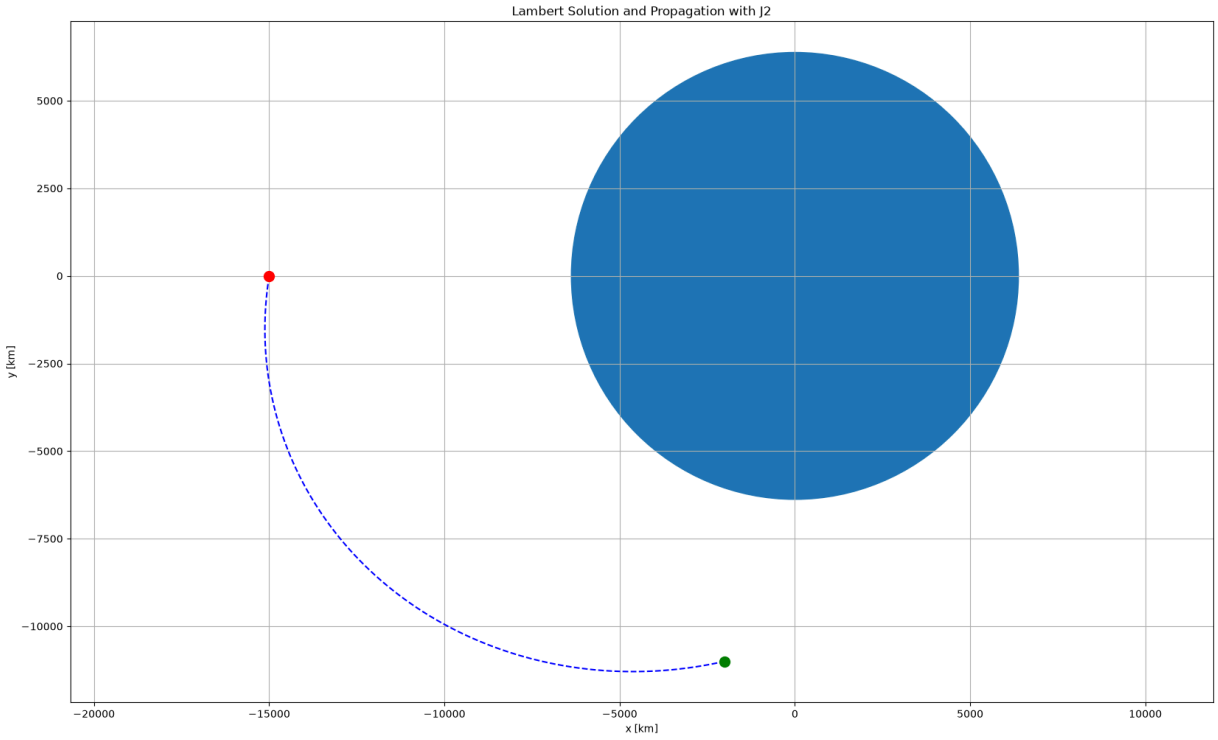
```
# Trajectory
ax.plot(sol_one_update[0], sol_one_update[1], 'b--', label='Trajectory')

# Points
ax.plot(r1[0], r1[1], 'ro', markersize=10, label='r1')
ax.plot(r2[0], r2[1], 'go', markersize=10, label='r2')

# Body
ax.add_patch(plt.Circle((0,0), R_EARTH, color='tab:blue', fill=True))


ax.set_xlabel('x [km]')
ax.set_ylabel('y [km]')
ax.set_aspect('equal', adjustable='datalim')

ax.grid(True)
plt.show()
```



Part f

In [20]:
```
state0_full_updated = find_initial_conditions(t_arr, two_body_with_J2, np.cc
v1_full_updated = state0_full_updated[3:6]
```

```
Converged in 3 iterations with:
       dr = 8.98310e-11 km
```

In [21]:
```
print(f"One-step v1 value [km/s]: \n\t{v1_one_update} ({norm(v1_one_update):
print(f"Final Updated v1 value [km/s]: \n\t{v1_full_updated} ({norm(v1_full_
print()
print(f"Difference [km/s]:\n\t{v1_one_update-v1_full_updated}, ({norm(v1_one
```

```
One-step v1 value [km/s]:
        [-6.55880979e-01 -4.06399725e+00  5.80741636e-31] (4.11658274 km/s)
Final Updated v1 value [km/s]:
        [-6.55880936e-01 -4.06399751e+00  5.40032976e-34] (4.11658300 km/s)

Difference [km/s]:
        [-4.34170663e-08  2.62738334e-07  5.80201603e-31], (2.663e-07 km/s)
```

AA 279B - Homework 3 Problem 3

Imports

In [1]:
```
%load_ext autoreload
%autoreload 2
```

In [2]:
```
import numpy as np
from numpy.linalg import norm
from scipy.integrate import solve_ivp
from scipy.optimize import fsolve
from scipy.interpolate import CubicSpline
import matplotlib.pyplot as plt
from lamberthub import vallado2013
import plotly.graph_objects as go
from tqdm import tqdm
```

Constants

In [3]:
```
RAD_TO_SIDEREAL_HOUR = 24.0/np.pi/2.0
MIN_TO_SEC = 60
HOUR_TO_SEC = 60 * MIN_TO_SEC
DAY_TO_SEC = 24 * HOUR_TO_SEC

M_TO_KM = 1.0/1000

DEG_TO_RAD = np.pi/180.0
RAD_TO_DEG = 1/DEG_TO_RAD

# km3/m2
MU_EARTH = 398600.4418

# km
R_EARTH = 6378.137

# rad/s
W_EARTH = 0.0000729211585530

MIN_TO_SEC = 60
```

In [4]:
```
# For sidereal

# 00:00 Jan 1 2014, UTC
t_utc_2014 = 1388534400
theta_utc_2014 = 6.7045617 / RAD_TO_SIDEREAL_HOUR

# Launcher data
launch_lon = -122.081944 * DEG_TO_RAD
launch_lat = 37.389444 * DEG_TO_RAD
```

```
launch_alt = 11 * M_TO_KM

# Dove 3 Orbital Elements
dove_epoch_day_num = 21.68857885
dove_epoch_utc = (dove_epoch_day_num - 1) * DAY_TO_SEC + t_utc_2014
i_dove = 97.7791 * DEG_TO_RAD
raan_dove = 95.3093 * DEG_TO_RAD
e_dove = 0.0155616
aop_dove = 357.8679 * DEG_TO_RAD
M_dove = 2.1812 * DEG_TO_RAD
n_dove = 14.56715041 / DAY_TO_SEC * 2 * np.pi # rev/day --> rad/s

a_dove = (MU_EARTH / n_dove/n_dove)**((1/3)) # Calc

# Sim start
# 0100h on 23 January 2014 UTC
sim_start_utc = 1390438800
```

Part a

In [5]:
```
def calc_theta_g(t_utc, omega = W_EARTH, theta_ref = theta_utc_2014, t_ref =
    """Simple rotation from Greenwich

    Parameters
    ----------
    t_utc : float
        seconds
    omega : float, optional
        default `W_EARTH`
    theta_ref : float, optional
        default `theta_utc_2014`
    t_ref : float, optional
        unix epoch of Jan 1, 2024, default `time_utc_2014`

    Returns
    ------
    theta_g : float
    """
    return theta_ref + omega * (t_utc - t_ref)
```

Part b

In [6]:
```
def kepler(M, e):
    """Vallado 4e eq. 2-4 p. 45 (pdf p. 72)"""
    E = fsolve(lambda E: E - e*np.sin(E) - M, M)[0]
    return E

def dcm_x(theta):
    """theta: angle of frame 2 wrt frame 1"""
    return np.array([
        [1, 0, 0],
        [0, np.cos(theta), np.sin(theta)],
        [0, -np.sin(theta), np.cos(theta)]
```

```
    ])

def dcm_y(theta):
    """theta: angle of frame 2 wrt frame 1"""
    return np.array([
        [np.cos(theta), 0, -np.sin(theta)],
        [0, 1, 0],
        [np.sin(theta), 0, np.cos(theta)]
    ])

def dcm_z(theta):
    """theta: angle of frame 2 wrt frame 1"""
    return np.array([
        [np.cos(theta), np.sin(theta), 0],
        [-np.sin(theta), np.cos(theta), 0],
        [0, 0, 1]
    ])


def rv_from_coe(a, e, i, raan, aop, M, mu = MU_EARTH) -> tuple[np.ndarray, r
    """Vallado 4e Algorithm 10 p. 118 (pdf p. 145)"""

    # semiparameter - Vallado 4e p. 96 (pdf p. 123)
    p = a*(1 - e*e)

    E = kepler(M,e)

    # true anomaly - Vallado 4e. eq. 2-10 p. 47 (pdf p. 74)
    nu = 2 * np.arctan2(
    np.sqrt(1 + e) * np.sin(E / 2),
    np.sqrt(1 - e) * np.cos(E / 2)
)
    r_pqw = np.array([
        p*np.cos(nu),
        p*np.sin(nu),
        0
    ]) / (1 + e*np.cos(nu))

    v_pqw = np.array([
        -np.sin(nu),
        e + np.cos(nu),
        0
    ]) * np.sqrt(mu/p)

    dcm_ijk_wrt_pqw = dcm_z(-raan) @ dcm_x(-i) @ dcm_z(-aop)
    r_ijk = dcm_ijk_wrt_pqw @ r_pqw
    v_ijk = dcm_ijk_wrt_pqw @ v_pqw

    return r_ijk, v_ijk


# Vallado 4e example 2-6 p. 119 (pdf p. 146)
assert np.allclose(
    np.array(rv_from_coe(
        a = 36126.6428348052,
        e = 0.83285,
```

```
        i = 87.87 * DEG_TO_RAD,
        raan = 227.89 * DEG_TO_RAD,
        aop = 53.38 * DEG_TO_RAD,
        M = 7.60494 * DEG_TO_RAD)),
    np.array([
        [6525.344, 6861.535, 6449.125],
        [4.902276, 5.533124, -1.975709]
    ])
)
```

Calculation

In [7]:
```
dt_launch = 15 # seconds
dt_tof = 15 # seconds
MIN_LAUNCH_DELAY = 0
MAX_LAUNCH_DELAY = 6 * HOUR_TO_SEC
MIN_TOF = 0
MAX_TOF = 2 * HOUR_TO_SEC

# # For earliest arrival
# dt_launch = 5 # seconds
# dt_tof = 5 # seconds
# MIN_LAUNCH_DELAY = 210 * MIN_TO_SEC
# MAX_LAUNCH_DELAY = 230 * MIN_TO_SEC
# MIN_TOF = 5 * MIN_TO_SEC
# MAX_TOF = 25 * MIN_TO_SEC

# # For smallest tof
# dt_launch = 5 # seconds
# dt_tof = 5 # seconds
# MIN_LAUNCH_DELAY = 320 * MIN_TO_SEC
# MAX_LAUNCH_DELAY = 340 * MIN_TO_SEC
# MIN_TOF = 1 * MIN_TO_SEC
# MAX_TOF = 10 * MIN_TO_SEC




t_arr_launch = np.arange(MIN_LAUNCH_DELAY, MAX_LAUNCH_DELAY + dt_launch, dt_
t_arr_tofs = np.arange(MIN_TOF, MAX_TOF + dt_tof, dt_tof)


N_LAUNCHES = len(t_arr_launch)
N_TOFS = len(t_arr_tofs)

print(f"Total launch delays: {N_LAUNCHES}")
print(f"Total tof: {N_TOFS}")
```

```
Total launch delays: 1441
Total tof: 481
```

Time grid / search space

In [8]:
```
launch_grid, tof_grid = np.meshgrid(
    t_arr_launch,
    t_arr_tofs,
    indexing="ij"
)

# dimensions of [x_dim, y_dim]
intercept_grid = launch_grid + tof_grid
print(intercept_grid.shape)

# From beginning of ephemeris time
t_total_utc = np.unique(np.concatenate(
        (np.array(sorted(intercept_grid.ravel())),t_arr_launch)
    )) + sim_start_utc # all places to evaluate

time_pairs = np.stack([
    launch_grid.ravel(),
    intercept_grid.ravel()
], axis=1)
```

```
(1441, 481)
```

In [9]:
```
fig = go.Figure()

# plotly expects [y_dim, x_dim] so z.T
fig.add_trace(go.Heatmap(
    x=t_arr_launch,
    y=t_arr_tofs,
    z=intercept_grid.T,
    colorscale="Viridis",
    colorbar=dict(title="Arrival time [s]"),
    opacity=0.8,
))

fig.add_trace(go.Contour(
    x=t_arr_launch,
    y=t_arr_tofs,
    z=intercept_grid.T,
    showscale=False,
    line=dict(width=2, color="black"),
    contours=dict(
        coloring="lines",
        showlabels=True,
        start=0,
        end=np.max(intercept_grid),
        size=2500,
        labelfont=dict(size=10, color="black")
    )
))

# fig.add_trace(go.Scatter(
#     x=launch_grid.ravel(),
#     y=tof_grid.ravel(),
#     mode="markers",
#     marker=dict(
#         size=4,
```

```
#         color="black",
#         opacity=0.7
#     ),
#     name="Grid points"
# ))

fig.update_layout(
    xaxis_title="Launch time [s]",
    yaxis_title="TOF [s]",
    title="Search Space",
    width=2000,
    height=500
)

fig.update_xaxes(
    range=[np.min(t_arr_launch), np.max(t_arr_launch)],
    constrain="domain"
)

fig.update_yaxes(
    range=[np.min(t_arr_tofs), np.max(t_arr_tofs)],
    constrain="domain"
)

fig.show()
```

In [10]:

```
def plot_3d(
    t_arrs,
    traces,
    labels="Orbit",
    title="3D Orbits",
    r_earth=R_EARTH,
    t_max=None
):

    fig = go.Figure()

    # Earth
    u_ = np.linspace(0, 2*np.pi, 80)
    v_ = np.linspace(0, np.pi, 40)

    x_earth = r_earth * np.outer(np.cos(u_), np.sin(v_))
    y_earth = r_earth * np.outer(np.sin(u_), np.sin(v_))
    z_earth = r_earth * np.outer(np.ones_like(u_), np.cos(v_))

    fig.add_trace(go.Surface(
        x=x_earth,
        y=y_earth,
        z=z_earth,
        name="Earth",
        showscale=False,
        opacity=0.7,
        colorscale=[[0, "royalblue"], [1, "royalblue"]]
    ))
```

```
    # Allow a single (N,3) trajectory
    if isinstance(traces, np.ndarray) and traces.ndim == 2:
        traces = [traces]

    # One time array shared by all trajectories
    if isinstance(t_arrs, np.ndarray) and t_arrs.ndim == 1:
        t_arrs = [t_arrs] * len(traces)

    if isinstance(labels, str):
        labels = [labels] * len(traces)

    for t_arr, trace, label in zip(t_arrs, traces, labels):
        t_arr = np.asarray(t_arr)
        trace = np.asarray(trace)

        if trace.ndim != 2 or trace.shape[1] != 3:
            raise ValueError(
                f"{label}: trace must have shape (N, 3), got {trace.shape}"
            )

        if len(t_arr) != len(trace):
            raise ValueError(
                f"{label}: len(t_arr)={len(t_arr)} != len(trace)={len(trace)
            )

        # Clip to requested maximum time
        if t_max is not None:
            mask = t_arr <= t_max
            t_arr = t_arr[mask]
            trace = trace[mask]

        if len(t_arr) < 2:
            continue

        cs = CubicSpline(t_arr, trace, axis=0)

        t_fine = np.linspace(t_arr[0], t_arr[-1], 1000)
        r_fine = cs(t_fine)

        fig.add_trace(go.Scatter3d(
            x=r_fine[:, 0],
            y=r_fine[:, 1],
            z=r_fine[:, 2],
            mode="lines",
            name=f"{label} Trajectory",
            line=dict(width=5)
        ))

        fig.add_trace(go.Scatter3d(
            x=[r_fine[-1, 0]],
            y=[r_fine[-1, 1]],
            z=[r_fine[-1, 2]],
            mode="markers",
            name=f"{label} Final position",
            marker=dict(size=6)
        ))
```

```
    fig.update_layout(
        title=title,
        scene=dict(
            xaxis_title="x [km]",
            yaxis_title="y [km]",
            zaxis_title="z [km]",
            aspectmode="data"
        ),
        template="plotly_white",
        legend=dict(x=0.02, y=0.98),
        height=900
    )

    fig.show()
```

Launch positions

In [11]:
```
"""
Inputs:
- Time
- Lat/lon

"""

def calc_ecef_from_lat_lon_alt(lat, lon, alt_km = 0, deg = False, r_body_km
    if deg:
        lat *= DEG_TO_RAD
        lon *= DEG_TO_RAD

    rho = r_body_km + alt_km

    r_ecef = np.array([
        rho * np.cos(lat) * np.cos(lon),
        rho * np.cos(lat) * np.sin(lon),
        rho * np.sin(lat)
    ])
    v_ecef = np.cross([0,0,W_EARTH], r_ecef)

    return r_ecef, v_ecef

def calc_eci_from_ecef(r_ecef, v_ecef, t_utc):
    """Somewhat simplified version of itrf-gcrs in Vallado 4e Section 3.7 pp
    # Earth rotation angle
    theta_ecef_wrt_eci = calc_theta_g(t_utc)

    dcm_eci_wrt_ecef = dcm_z(theta_ecef_wrt_eci).T

    r_eci = dcm_eci_wrt_ecef @ r_ecef
    v_eci = dcm_eci_wrt_ecef @ v_ecef

    return r_eci, v_eci
```

In [12]:
```
## Actual calculation

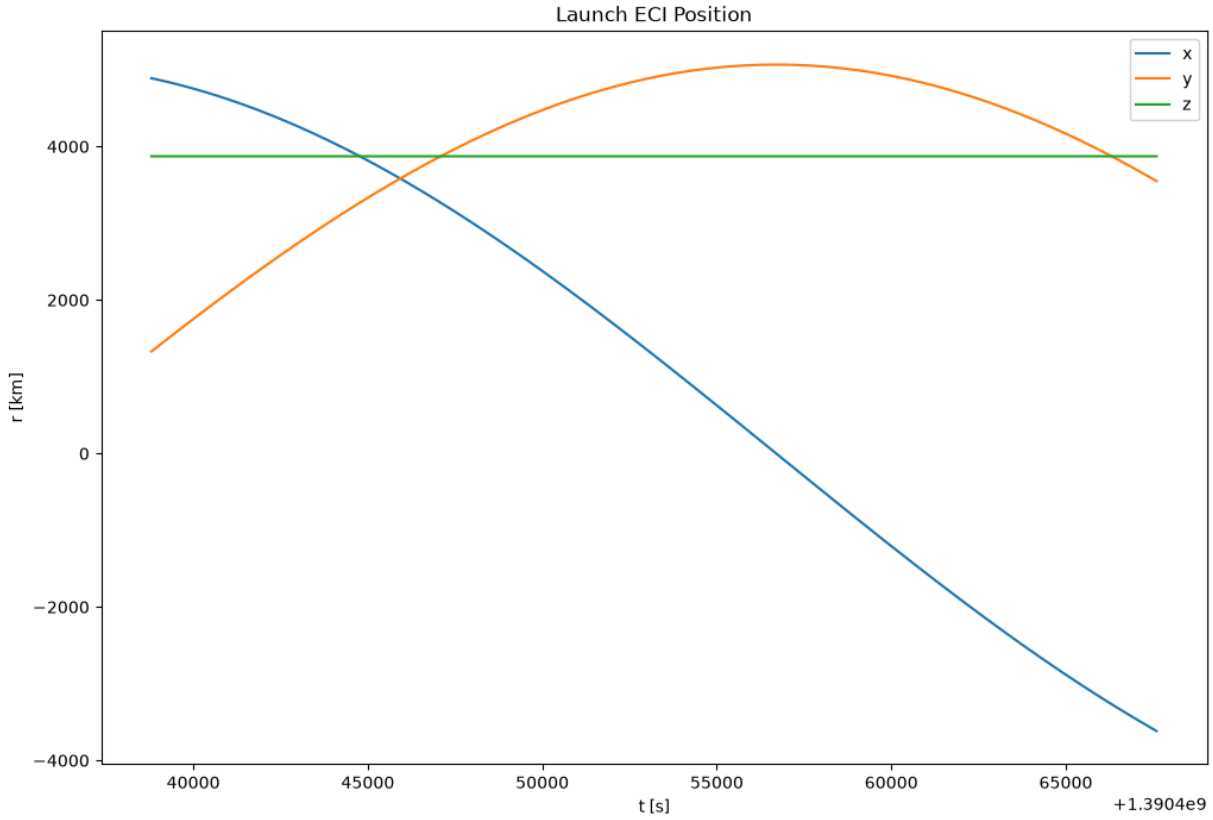
def launch_eci(t_arr_utc, lat, lon, alt):
    launch_r_ecef, launch_v_ecef = calc_ecef_from_lat_lon_alt(lat, lon, alt)

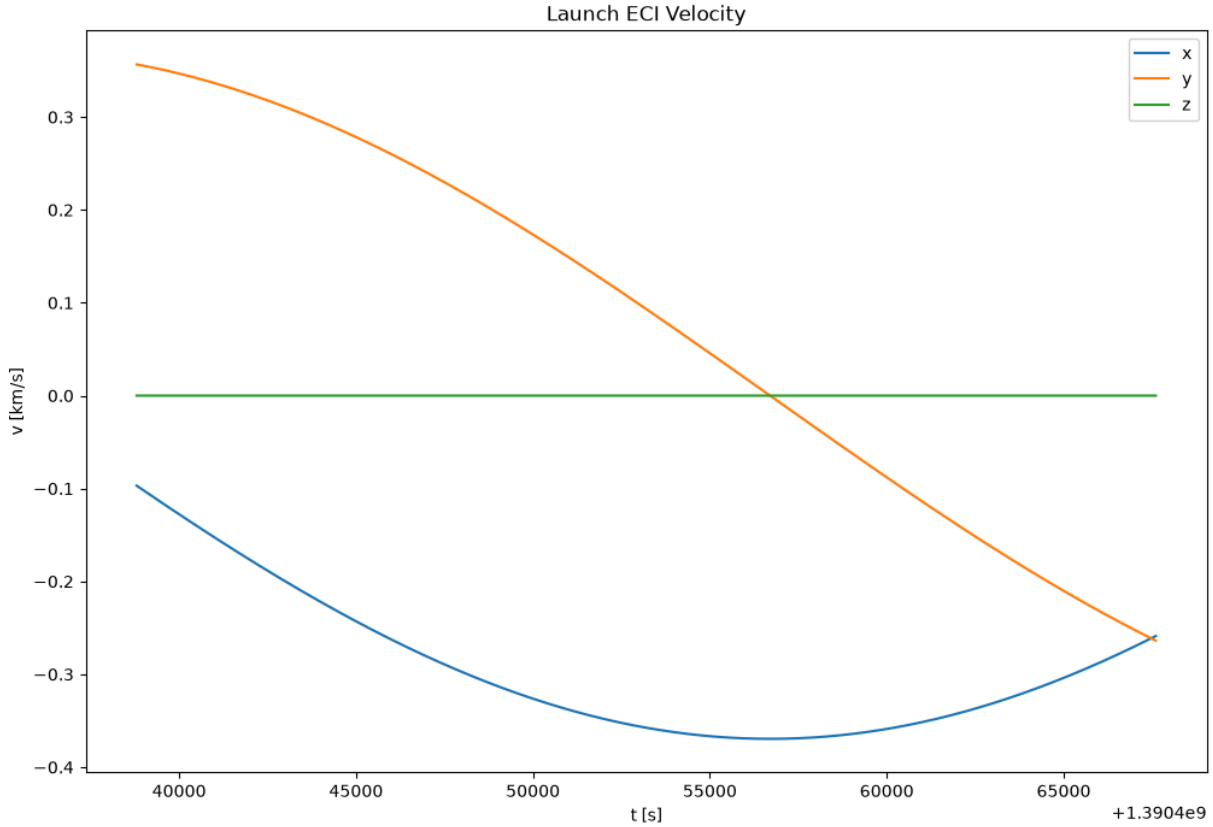
    launch_arr_eci = np.array([calc_eci_from_ecef(launch_r_ecef, launch_v_ec
    launch_r_eci = launch_arr_eci[:,0,:]
    launch_v_eci = launch_arr_eci[:,1,:]
    return launch_r_eci, launch_v_eci
```

In [13]:
```
launch_r_eci, launch_v_eci = launch_eci(t_total_utc, launch_lat, launch_lon,

plt.figure(figsize=(12,8))
plt.title("Launch ECI Position")
plt.xlabel("t [s]")
plt.ylabel("r [km]")
plt.plot(t_total_utc, launch_r_eci[:,0], label = "x")
plt.plot(t_total_utc, launch_r_eci[:,1], label = "y")
plt.plot(t_total_utc, launch_r_eci[:,2], label = "z")
plt.legend()
plt.show()

plt.figure(figsize=(12,8))
plt.title("Launch ECI Velocity")
plt.xlabel("t [s]")
plt.ylabel("v [km/s]")
plt.plot(t_total_utc, launch_v_eci[:,0], label = "x")
plt.plot(t_total_utc, launch_v_eci[:,1], label = "y")
plt.plot(t_total_utc, launch_v_eci[:,2], label = "z")
plt.legend()
plt.show()
```





In [14]:
```
plot_3d(t_total_utc, launch_r_eci, "Launcher", title="3D Launch Site Positic
```

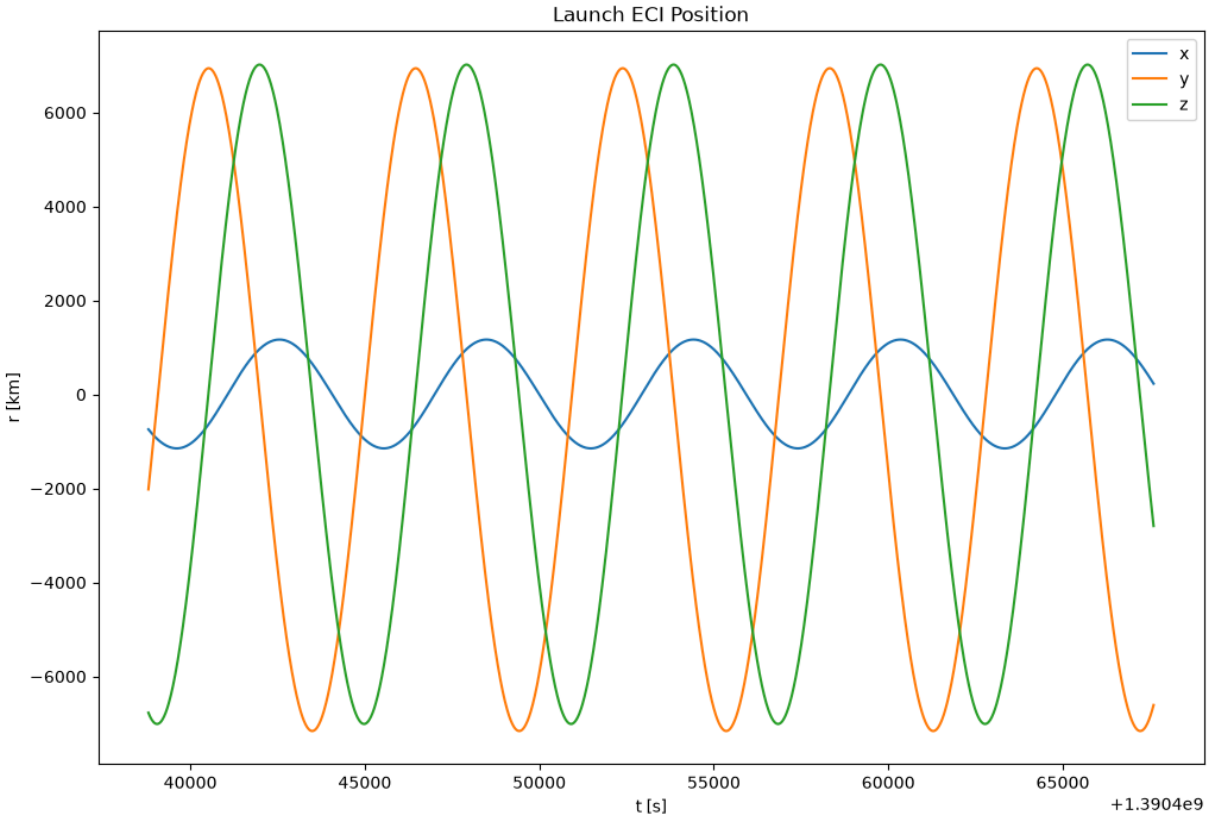
Sat positions

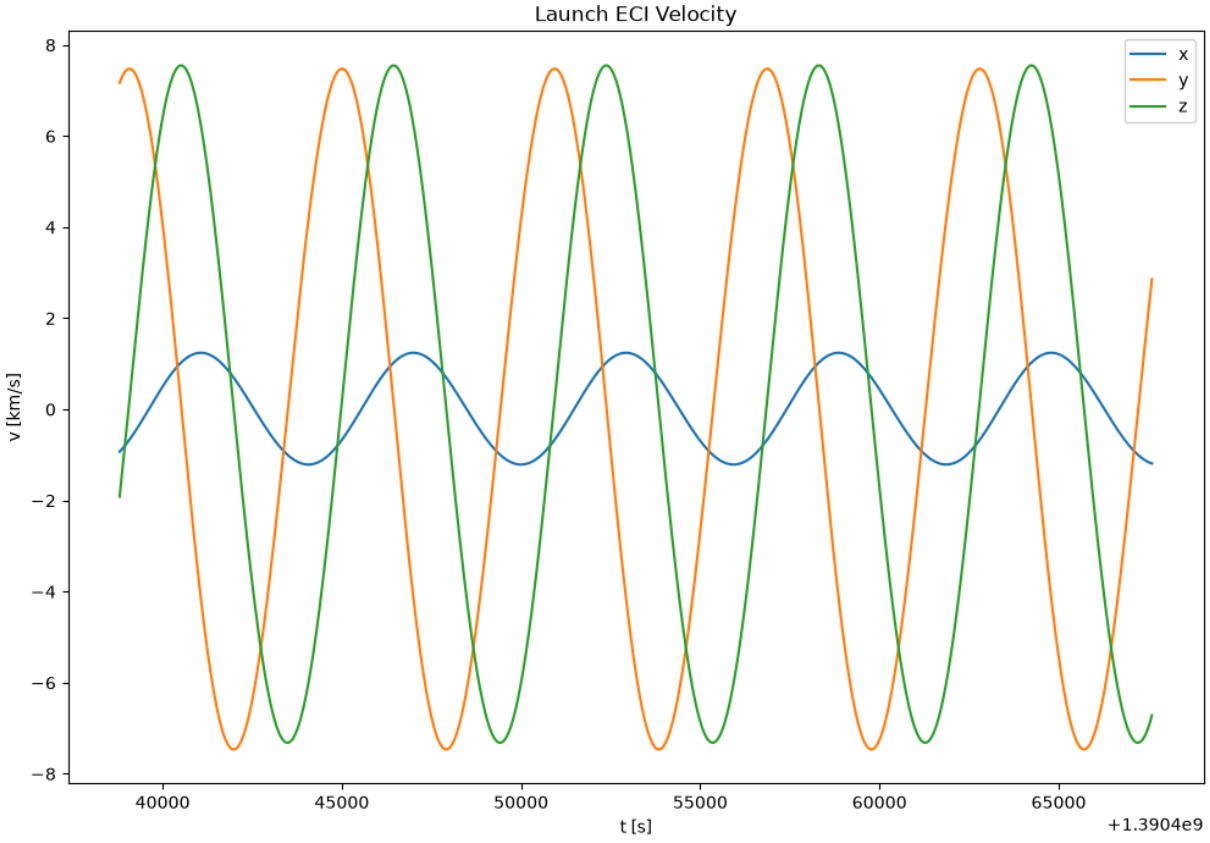
In [15]:
```
## Sat functions
def dove3_rv_eci(t_arr, t0 = dove_epoch_utc) -> np.ndarray | tuple[np.ndarra
    # M_new = (M_dove + n_dove*(t_arr - t0))
    M_new = np.mod(
        M_dove + n_dove*(t_arr - t0), 2*np.pi
    )
    rv = np.array([rv_from_coe(a_dove,e_dove,i_dove,raan_dove,aop_dove,M) fo
    r = rv[:,0,:]
    v = rv[:,1,:]
    return r,v

dove3_r_eci, dove3_v_eci = dove3_rv_eci(t_total_utc)

plt.figure(figsize=(12,8))
plt.title("Launch ECI Position")
plt.xlabel("t [s]")
plt.ylabel("r [km]")
plt.plot(t_total_utc, dove3_r_eci[:,0], label = "x")
plt.plot(t_total_utc, dove3_r_eci[:,1], label = "y")
plt.plot(t_total_utc, dove3_r_eci[:,2], label = "z")
plt.legend()
plt.show()

plt.figure(figsize=(12,8))
plt.title("Launch ECI Velocity")
plt.xlabel("t [s]")
plt.ylabel("v [km/s]")
plt.plot(t_total_utc, dove3_v_eci[:,0], label = "x")
plt.plot(t_total_utc, dove3_v_eci[:,1], label = "y")
plt.plot(t_total_utc, dove3_v_eci[:,2], label = "z")
plt.legend()
plt.show()
```





Both positions

In [16]:
```
plot_3d(t_total_utc, [launch_r_eci, dove3_r_eci], ["Launcher", "Dove 3"], ti
```

Calculation

In [17]:
```
# Search space
dv1_arr = np.full((N_LAUNCHES, N_TOFS), np.nan)
dv2_arr = np.full((N_LAUNCHES, N_TOFS), np.nan)
collision_arr = np.full((N_LAUNCHES, N_TOFS), np.nan)
hyper_para_arr = np.full((N_LAUNCHES, N_TOFS), np.nan)
err_arr = np.full((N_LAUNCHES, N_TOFS), np.nan)
print(f"Search space: {dv1_arr.shape}, with {N_LAUNCHES * N_TOFS} total")
```

```
Search space: (1441, 481), with 693121 total
```

In [18]:
```
# def find_launch_rv(t,t_arr,r_arr,v_arr):
#     i = np.argmin(np.abs(t_arr - t))
#     return r_arr[i], v_arr[i]

def find_rv(t, t_arr, r_arr, v_arr):
    i = np.searchsorted(t_arr, t)
    assert np.isclose(t_arr[i], t)
    return r_arr[i], v_arr[i]

for launch_num in tqdm(range(N_LAUNCHES)):
    t_launch = t_arr_launch[launch_num]
    # print(f"Launch offset: {t_launch}")
    for tof_num in range(N_TOFS):
        tof = t_arr_tofs[tof_num]
        t_intercept = t_launch + tof
        # print(f"\tIntercept time: {t_intercept}")


        r_launch, v_launch = find_rv(t_launch + sim_start_utc, t_total_utc,
        r_sat, v_sat = find_rv(t_intercept + sim_start_utc, t_total_utc, dov

        # Parabola/hyperbola check (short way)
        c = norm(r_launch - r_sat)
        s = (norm(r_launch) + norm(r_sat) + c) / 2
        dt_parabola = 1/3 * np.sqrt(2/MU_EARTH) * (s**(3/2) - (s-c)**(3/2))
        if tof <= dt_parabola:
            hyper_para_arr[launch_num, tof_num] = 1
            continue

        # Call solver
        try:
            cross_z = np.cross(r_launch, r_sat)[2]

            # Allows satellite to go retrograde if it is "behind"/west of th
            prograde = cross_z >= 0

            v1, v2 = vallado2013(
                MU_EARTH,
                r_launch,
                r_sat,
                tof,
                # M=1,  # No multi-orbit solutions
                prograde=prograde,
```

```
                low_path=True                    # short way
            )
        except ValueError:
            err_arr[launch_num, tof_num] = 1
            continue


        # Earth-impact check
        if np.dot(r_launch, v1) < 0 and np.dot(r_sat, v2) > 0:
            e_sp = 1/2 * norm(v1)**2 - MU_EARTH / norm(r_launch)
            a = -MU_EARTH / 2 / e_sp
            h = norm(np.cross(r_launch, v1))
            p = h*h/MU_EARTH
            e = np.sqrt((a-p)/a)
            r_p = a*(1-e)

            if r_p <= R_EARTH:
                collision_arr[launch_num, tof_num] = 1
                continue

        dv1 = norm(v1 - v_launch)
        dv2 = norm(v_sat - v2)
        # print()



        dv1_arr[launch_num, tof_num] = dv1
        dv2_arr[launch_num, tof_num] = dv2

dv_arr = dv1_arr + dv2_arr
```

```
100%|██████████| 1441/1441 [01:20<00:00, 18.00it/s]
```

Part c/d

In [19]:

```
fig = go.Figure()

# plotly expects [y_dim, x_dim] so z.T
fig.add_trace(go.Heatmap(
    x=t_arr_launch / MIN_TO_SEC,
    y=t_arr_tofs / MIN_TO_SEC,
    z=dv1_arr.T,
    colorscale="Viridis",
    colorbar=dict(title="Departure dv [km/s]"),
    showscale=False,
    opacity=0.8,
))

fig.add_trace(go.Contour(
    x=t_arr_launch / MIN_TO_SEC,
    y=t_arr_tofs / MIN_TO_SEC,
    z=dv1_arr.T,

    contours=dict(
        # start=3.5,
```

```
        # end=12,
        # size=0.5,
        coloring="fill",
        showlabels=True,
    ),

    line=dict(
        width=1,
        smoothing=1.3
    ),

    colorscale="Viridis",
    colorbar=dict(title="Departure dv [km/s]"),
))


# fig.add_trace(go.Scatter(
#     x=launch_grid.ravel(),
#     y=tof_grid.ravel(),
#     mode="markers",
#     marker=dict(
#         size=4,
#         color="black",
#         opacity=0.7
#     ),
#     name="Grid points"
# ))

fig.update_layout(
    title="Total dV for intercept",
    xaxis_title="Launch time after sim start [min]",
    yaxis_title="Time of flight [min]",
    width=1200,
    height=700
)

fig.update_xaxes(
    range=[np.min(t_arr_launch / MIN_TO_SEC), np.max(t_arr_launch / MIN_TO_S
    constrain="domain"
)

fig.update_yaxes(
    range=[np.min(t_arr_tofs / MIN_TO_SEC), np.max(t_arr_tofs / MIN_TO_SEC)]
    constrain="domain"
)

fig.write_image("hw3_plots_test/15_sec_total_dv_plot.png", width=2000, heigh


fig.show()
```

In [20]:
```
print("The dv plot seems to have a period of about 90 minutes (typical of a
print("The dv in the center of its bulbs also decreases as time goes on.")
```

The dv plot seems to have a period of about 90 minutes (typical of a LEO satellite).
The dv in the center of its bulbs also decreases as time goes on.

In [21]:
```
fig = go.Figure()

# plotly expects [y_dim, x_dim] so z.T
fig.add_trace(go.Heatmap(
    x=t_arr_launch / MIN_TO_SEC,
    y=t_arr_tofs / MIN_TO_SEC,
    z=dv_arr.T,
    colorscale="Viridis",
    colorbar=dict(title="Departure dv [km/s]"),
    showscale=False,
    opacity=0.8,
))

fig.add_trace(go.Contour(
    x=t_arr_launch / MIN_TO_SEC,
    y=t_arr_tofs / MIN_TO_SEC,
    z=dv_arr.T,

    contours=dict(
        # start=3.5,
        # end=12,
        # size=0.5,
        coloring="fill",
        showlabels=True,
    ),

    line=dict(
        width=1,
        smoothing=1.3
    ),

    colorscale="Viridis",
    colorbar=dict(title="Departure dv [km/s]"),
))


# fig.add_trace(go.Scatter(
#     x=launch_grid.ravel(),
#     y=tof_grid.ravel(),
#     mode="markers",
#     marker=dict(
#         size=4,
#         color="black",
#         opacity=0.7
#     ),
#     name="Grid points"
# ))

fig.update_layout(
    title="Total dV for rendezvous",
    xaxis_title="Launch time after sim start [min]",
    yaxis_title="Time of flight [min]",
```

```
    width=1200,
    height=700
)

fig.update_xaxes(
    range=[np.min(t_arr_launch / MIN_TO_SEC), np.max(t_arr_launch / MIN_TO_S
    constrain="domain"
)

fig.update_yaxes(
    range=[np.min(t_arr_tofs / MIN_TO_SEC), np.max(t_arr_tofs / MIN_TO_SEC)]
    constrain="domain"
)

fig.write_image("hw3_plots_test/15_sec_rpo_dv_plot.png", width=2000, height=


fig.show()
```

In [22]:
```
print("Looks very similar to the intercept plot, but there are clear diagona
print("The total dv required in the center of the bulbs also decreases each
print("The lines where the solution becomes invalid (intercepts Earch or par
print("This is likely because the optimal trajectory goes North slowly near
```

```
Looks very similar to the intercept plot, but there are clear diagonal lines
that have a lot of dv.
The total dv required in the center of the bulbs also decreases each period.
The lines where the solution becomes invalid (intercepts Earch or parabolic/
hyperbolic) are almost vertical close to the minimum dv.
This is likely because the optimal trajectory goes North slowly near the sat
ellite's path, so delaying even a bit more will cause it to miss entirely.
```

Part e/f

3D Optimal Plots

In [23]:
```
# Finding min

def min_dv(dv_grid, launch_grid, tof_grid):
    dv_min = np.nanmin(dv_grid)
    idx_grid = np.nanargmin(dv_grid)
    launch_idx, tof_idx = np.unravel_index(idx_grid, dv_grid.shape)

    return dv_min, launch_grid[launch_idx, tof_idx], tof_grid[launch_idx, to

dv_min_int, launch_delay_int_s, tof_int_s = min_dv(dv1_arr, launch_grid, tof
print(f"Min dv for intercept:\n\t{dv_min_int:.3f} km/s at delay of {launch_d

dv_min_rpo, launch_delay_rpo_s, tof_rpo_s = min_dv(dv_arr, launch_grid, tof_
print(f"Min dv for rendezvous:\n\t{dv_min_rpo:.3f} km/s at delay of {launch_
```

```
Min dv for intercept:
        3.860 km/s at delay of 326.0 min and tof of 8.75 min
Min dv for rendezvous:
        8.984 km/s at delay of 333.75 min and tof of 19.75 min
```

In [24]:
```
print("It could be less than the circular velocity because the launcher has
```

```
It could be less than the circular velocity because the launcher has an init
ial velocity due to the rotation of the Earth
```

In [25]:
```
# Optimal trajectory intercept

def two_body(t,x,mu=MU_EARTH):
    del t
    r,v = x[0:3], x[3:6]
    a = -mu / norm(r)**3 * r
    return np.concat((v,a))

def propagate_traj(launch_delay_s, tof_s):
    r_launch, _ = find_rv(launch_delay_s + sim_start_utc, t_total_utc, launc
    r_sat, _ = find_rv(tof_s + launch_delay_s + sim_start_utc, t_total_utc,

    cross_z = np.cross(r_launch, r_sat)[2]
    prograde = cross_z >= 0

    v1, _ = vallado2013(
        MU_EARTH,
        r_launch,
        r_sat,
        tof_s,
        prograde=prograde,
        low_path=True                    # short way
    )

    t_optimal_propagation_arr = np.linspace(0, tof_s, 1000)

    sol_intercept: np.ndarray = solve_ivp(two_body, (0, tof_s), np.concatena

    opt_t_utc = sim_start_utc + launch_delay_s + sol_intercept.t
    return opt_t_utc, sol_intercept.y

opt_int_t_utc, opt_int_r_arr = propagate_traj(launch_delay_int_s, tof_int_s)
opt_rpo_t_utc, opt_rpo_r_arr = propagate_traj(launch_delay_rpo_s, tof_rpo_s)
```

In [26]:
```
plot_3d([t_total_utc, t_total_utc, opt_int_t_utc], [launch_r_eci, dove3_r_ec
```

In [27]:
```
plot_3d([t_total_utc, t_total_utc, opt_rpo_t_utc], [launch_r_eci, dove3_r_ec
```

Part g/h

In [28]:
```
MAX_DV = 5 # km/s
```

```
dv_limited = dv1_arr.copy()
dv_limited[dv_limited > MAX_DV] = np.nan
```

In [29]:
```
def earliest_time_dv(dv_grid, launch_grid, tof_grid):
    intercept_grid = (launch_grid + tof_grid).astype(float)
    intercept_grid[np.isnan(dv_grid)] = np.nan
    intercept_idx = np.nanargmin(intercept_grid)
    launch_idx, tof_idx = np.unravel_index(intercept_idx, intercept_grid.sha

    dv_min = dv_grid[launch_idx, tof_idx]

    return dv_min, launch_grid[launch_idx, tof_idx], tof_grid[launch_idx, tc

def smallest_tof_dv(dv_grid, launch_grid, tof_grid):
    new_tof_grid = tof_grid.copy().astype(float)
    new_tof_grid[np.isnan(dv_grid)] = np.nan
    idx = np.nanargmin(new_tof_grid)
    launch_idx, tof_idx = np.unravel_index(idx, intercept_grid.shape)

    dv_min = dv_grid[launch_idx, tof_idx]

    return dv_min, launch_grid[launch_idx, tof_idx], tof_grid[launch_idx, tc


dv_early, launch_delay_early, tof_delay_early = earliest_time_dv(dv_limited,
print(f"Quickest intercept at:\n\t{(launch_delay_early + tof_delay_early) /

dv_small_tof, launch_delay_small_tof, tof_delay_small_tof = smallest_tof_dv(
print(f"Smallest tof at:\n\t{(launch_delay_small_tof + tof_delay_small_tof)
```

```
Quickest intercept at:
        232.25 min with delay of 216.25, tof of 16.0 min, and dv of 4.999 k
m/s
Smallest tof at:
        334.25 min with delay of 329.75, tof of 4.5 min, and dv of 4.929 km/
s
```

In [30]:
```
from datetime import datetime, timezone
# In UTC

dt_early_launch_utc = datetime.fromtimestamp(launch_delay_early + sim_start_
print(f"Launch time for quickest intercept:\n\t{dt_early_launch_utc} ({dt_ea

dt_smallest_tof_launch_utc = datetime.fromtimestamp(launch_delay_small_tof +
print(f"Launch time for smallest time of flight:\n\t{dt_smallest_tof_launch_
```

```
Launch time for quickest intercept:
        2014-01-23 04:36:15+00:00 (1390451775.0 in unix)
Launch time for smallest time of flight:
        2014-01-23 06:29:45+00:00 (1390458585.0 in unix)
```

In [31]:
```
fig = go.Figure()

fig.add_trace(go.Heatmap(
    x=t_arr_launch / MIN_TO_SEC,
    y=t_arr_tofs / MIN_TO_SEC,
    z=dv_limited.T,
```

```
    colorscale="Viridis",
    colorbar=dict(title="Departure dv [km/s]"),
    opacity=0.8
))

# fig.add_trace(go.Contour(
#     x=t_arr_launch / MIN_TO_SEC,
#     y=t_arr_tofs / MIN_TO_SEC,
#     z=intercept_grid.T / MIN_TO_SEC,
#     autocontour=False,
#     showscale=False,
#     showlegend=False,
#     line=dict(width=2, color="black"),
#     colorscale=[
#         [0, "blue"],
#         [1, "blue"]
#     ],

#     contours=dict(
#         coloring="lines",
#         showlabels=True,
#         start=(launch_delay_early + tof_delay_early)/MIN_TO_SEC,
#         end=(launch_delay_early + tof_delay_early)/MIN_TO_SEC,
#         labelfont=dict(size=10, color="black"),
#     ),
# ))

# fig.add_trace(go.Scatter(
#     x=[None], y=[None],
#     mode="lines",
#     line=dict(color="blue", width=2),
#     name=f"Minimum intercept time ({int((launch_delay_early + tof_delay_ea
# ))


# fig.add_trace(go.Contour(
#     x=t_arr_launch / MIN_TO_SEC,
#     y=t_arr_tofs / MIN_TO_SEC,
#     z=tof_grid.T / MIN_TO_SEC,
#     autocontour=False,
#     showscale=False,
#     showlegend=False,
#     line=dict(width=2, color="black"),
#     colorscale=[
#         [0, "orange"],
#         [1, "orange"]
#     ],

#     contours=dict(
#         coloring="lines",
#         showlabels=True,
#         start=tof_delay_small_tof / MIN_TO_SEC,
#         end=tof_delay_small_tof / MIN_TO_SEC,
#         labelfont=dict(size=10, color="black"),
#     ),
# ))
```

```
# fig.add_trace(go.Scatter(
#     x=[None], y=[None],
#     mode="lines",
#     line=dict(color="orange", width=2),
#     name=f"Minimum time of flight ({int(tof_delay_small_tof / MIN_TO_SEC)}
# ))

# Vibes
# fig.update_xaxes(
#     range=[200, 340],
#     constrain="domain"
# )

# fig.update_yaxes(
#     range=[3, 25],
#     constrain="domain"
# )

# fig.add_trace(go.Scatter(
#     # x=[launch_delay_early/MIN_TO_SEC],
#     # y=[tof_delay_early/MIN_TO_SEC],
#     x=[launch_delay_small_tof/MIN_TO_SEC],
#     y=[tof_delay_small_tof/MIN_TO_SEC],
#     mode="markers",
#     marker=dict(
#         symbol="x",
#         size=10,
#         color="red",
#     ),
#     name="Optimal point"
# ))

fig.update_layout(
    # title="Intercept with earliest time with dV < 5 km/s",
    # title="Intercept with smallest tof with dV < 5 km/s",
    title="dV < 5 km/s",
    xaxis_title="Launch time after sim start [min]",
    yaxis_title="Time of flight [min]",
    width=1200,
    height=700,
    legend=dict(
        x=0.02,
        y=0.98,
        xanchor="left",
        yanchor="top"
    )
)
# fig.write_image(f"hw3_plots/earliest_intercept_5_sec.png", width=2000, hei
# fig.write_image("hw3_plots_test/smallest_tof_5_sec.png", width=2000, heigh
fig.write_image("hw3_plots_test/earliest_smallest_15_sec.png", width=2000, h

fig.show()
```